

Software Engineer — Step 5

System Design Basics

Roadmap objective: Understand how larger software systems are structured and scaled

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Requirements
- HLD vs LLD
- Load balancing
- Caching
- Databases
- Queues
- CDN
- Scalability
- Availability
- Consistency
- Observability
- Security

Core concepts

1. High-level design focuses on major components and interactions; low-level design focuses on detailed component/class/API behavior.
2. Scaling can be vertical or horizontal; distributed systems introduce trade-offs rather than free performance.
3. Caching can reduce latency and database load but creates invalidation and consistency challenges.
4. A system design answer should state requirements and trade-offs before drawing boxes.

Concept map

```
System Design Basics
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Requirements

Study requirements through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. HLD vs LLD

Study hld vs lld through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Load balancing

Study load balancing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Caching

Study caching through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Databases

Study databases through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Queues

Study queues through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

System design flow

```
Client
|
v
Load Balancer
|
+----> App Server 1 ----+
+----> App Server 2 ----+--> Cache --> Database
|
+-----> Queue --> Worker

Ask:
```

- What traffic?
- What latency?
- What availability?
- What data consistency?
- What failure modes?

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Vertical scaling	Bigger machine	Simple, finite ceiling
Horizontal scaling	More instances	Requires distribution
Cache	Fast temporary copy	Invalidation challenge

Common mistakes

- Starting with technology names instead of requirements
- Ignoring bottlenecks
- No capacity assumptions
- Ignoring failures
- Drawing architecture without explaining trade-offs

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Pick a system such as a URL shortener.
- Write down what you expected, what happened, and why.

Lab 2

- Write functional requirements.
- Write down what you expected, what happened, and why.

Lab 3

- Estimate rough request volume.
- Write down what you expected, what happened, and why.

Lab 4

- Choose storage and cache strategy.
- Write down what you expected, what happened, and why.

Lab 5

- Identify one bottleneck and one failure mode.
- Write down what you expected, what happened, and why.

Lab 6

- Explain at least two alternative designs and their trade-offs.
- Write down what you expected, what happened, and why.

Mini-project

Design a URL shortener or career-platform backend on paper. State requirements, estimate traffic, choose components, discuss caching/database/queues and explain failure modes.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

HLD vs LLD?

- Answer structure: definition → example → trade-off/limitation → practical use.

Horizontal vs vertical scaling?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is caching?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why use a queue?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you discuss consistency?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you design for failure?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
HLD	Write this in your own words.	Give one project example.
LLD	Write this in your own words.	Give one project example.
Scalability	Write this in your own words.	Give one project example.
Availability	Write this in your own words.	Give one project example.
Latency	Write this in your own words.	Give one project example.
Cache	Write this in your own words.	Give one project example.
Load balancer	Write this in your own words.	Give one project example.
Queue	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Consistency	Write this in your own words.	Give one project example.
Observability	Write this in your own words.	Give one project example.

Self-test

- Explain Requirements without using its textbook definition.
- Show a small example of HLD vs LLD.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Requirements.
- I can explain and demonstrate HLD vs LLD.
- I can explain and demonstrate Load balancing.
- I can explain and demonstrate Caching.
- I can explain and demonstrate Databases.
- I can explain and demonstrate Queues.
- I can explain and demonstrate CDN.
- I can explain and demonstrate Scalability.
- I can explain and demonstrate Availability.
- I can explain and demonstrate Consistency.
- I can explain and demonstrate Observability.
- I can explain and demonstrate Security.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://aws.amazon.com/architecture/>
- <https://sre.google/books/>

Recommended videos

- Apna College — Complete System Design Roadmap 2025 | HLD & LLD
<https://www.youtube.com/watch?v=CuQmQpvw04I> (NEW)
- System Design Fundamentals (Existing DB resource 120)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.